

Tracing Problem: Write down the output of the following output and what the code is doing!

```
class Node:
    def __init__(self, data=0, next_node=None):
        self.data = data
        self.next = next_node

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        if not self.head:
            self.head = Node(data)
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = Node(data)

    def print_list(self):
        nodes = []
        current = self.head
        while current:
            nodes.append(str(current.data))
            current = current.next
        print(" -> ".join(nodes))

def add_lists(l1, l2, carry=0):
    if l1 is None and l2 is None and carry == 0:
        return None
    result = Node()
    value = carry
    if l1 is not None:
        value += l1.data
    if l2 is not None:
        value += l2.data
    result.data = value % 10
    if l1 is not None or l2 is not None or value >= 10:
        next_l1 = l1.next if l1 is not None else None
        next_l2 = l2.next if l2 is not None else None
        new_carry = 1 if value >= 10 else 0
```

```

        more = add_lists(next_l1, next_l2, new_carry)
        result.next = more
    return result

# --- Example Usage ---
if __name__ == "__main__":
    l11 = LinkedList()
    l11.append(7)
    l11.append(1)
    l11.append(6)

    l12 = LinkedList()
    l12.append(5)
    l12.append(9)
    l12.append(2)

    print("List 1:")
    l11.print_list()

    print("List 2:")
    l12.print_list()

    sum_head = add_lists(l11.head, l12.head)

    sum_ll = LinkedList()
    sum_ll.head = sum_head

    print("\nSum List:")
    sum_ll.print_list()

```

Answer:

List 1:

7 -> 1 -> 6

List 2:

5 -> 9 -> 2

Sum List:

2 -> 1 -> 9

Programming (Easy)

The following code intends to implement a function to check if a linked list is a palindrome. Complete the code.

```
class Node:
    """A simple node for a singly linked list."""
    def __init__(self, data=0, next_node=None):
        self.data = data
        self.next = next_node

class LinkedList:
    """A simple linked list class for demonstration purposes."""
    def __init__(self):
        self.head = None

    def append(self, data):
        """Appends a new node with the given data to the end of the list."""
        if not self.head:
            self.head = Node(data)
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = Node(data)

    def print_list(self):
        """Prints the elements of the list."""
        nodes = []
        current = self.head
        while current:
            nodes.append(str(current.data))
            current = current.next
        print(" -> ".join(nodes))

def reverse_and_clone(node):
    """
    This function intends to reverse and clone a linked list.

    Args:
        node: The head node of the linked list to reverse and clone.
    """
```

```
Returns:
    The head node of the new, reversed linked list.
```

COMPLETE THE FUNCTION

```
"""
```

```
def is_equal(one, two):
    """
    This function intends to compare two linked lists to see if they are identical.
```

```
Args:
```

```
    one: The head node of the first linked list.
    two: The head node of the second linked list.
```

```
Returns:
```

```
    True if the lists are equal, False otherwise.
```

COMPLETE THIS FUNCTION

```
"""
```

```
def is_palindrome(head):
    reversed_head = reverse_and_clone(head)
    return is_equal(head, reversed_head)
```

Answer:

```
class Node:
    """A simple node for a singly linked list."""
    def __init__(self, data=0, next_node=None):
        self.data = data
        self.next = next_node

class LinkedList:
    """A simple linked list class for demonstration purposes."""
    def __init__(self):
        self.head = None

    def append(self, data):
        """Appends a new node with the given data to the end of the list."""
        if not self.head:
```

```

        self.head = Node(data)
        return
    current = self.head
    while current.next:
        current = current.next
    current.next = Node(data)

def print_list(self):
    """Prints the elements of the list."""
    nodes = []
    current = self.head
    while current:
        nodes.append(str(current.data))
        current = current.next
    print(" -> ".join(nodes))

def reverse_and_clone(node):
    """
    Reverses and clones a linked list.

    Args:
        node: The head node of the linked list to reverse and clone.

    Returns:
        The head node of the new, reversed linked list.
    """
    new_head = None
    while node is not None:
        # Clone the node by creating a new one with the same data
        n = Node(node.data)
        # Prepend the new node to the reversed list
        n.next = new_head
        new_head = n
        # Move to the next node in the original list
        node = node.next
    return new_head

def is_equal(one, two):
    """
    Compares two linked lists to see if they are identical.

    Args:

```

```

    one: The head node of the first linked list.
    two: The head node of the second linked list.

Returns:
    True if the lists are equal, False otherwise.
"""
while one is not None and two is not None:
    if one.data != two.data:
        return False
    one = one.next
    two = two.next

# If both lists are now empty, they were the same length and are equal.
# If one is empty and the other isn't, they are not equal.
return one is None and two is None

def is_palindrome(head):
    """
    Checks if a linked list is a palindrome by reversing and cloning it,
    then comparing the original to the reversed copy.

    Args:
        head: The head node of the linked list.

    Returns:
        True if the linked list is a palindrome, False otherwise.
    """
    reversed_head = reverse_and_clone(head)
    return is_equal(head, reversed_head)

```

Programming (Hard)

We want to write code to partition a linked list around a value x , such that all nodes less than x come before all nodes greater than or equal to x . If x is contained within the list, the values of x only need to be after the elements less than x (see below). The partition element x can appear anywhere in the "right partition"; it does not need to appear between the left and right partitions.

EXAMPLE

Input: 3 -> 5 -> 8 -> 5 -> 10 -> 2 -> 1 [partition = 5]

Output: 3 -> 1 -> 2 -> 10 -> 5 -> 5 -> 8

Complete the code:

```
class Node:
    """A simple node for a singly linked list."""
    def __init__(self, data=0, next_node=None):
        self.data = data
        self.next = next_node

class LinkedList:
    """A simple linked list class for demonstration purposes."""
    def __init__(self):
        self.head = None

    def append(self, data):
        """Appends a new node with the given data to the end of the list."""
        if not self.head:
            self.head = Node(data)
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = Node(data)

    def print_list(self):
        """Prints the elements of the list."""
        nodes = []
        current = self.head
        while current:
            nodes.append(str(current.data))
            current = current.next
        print(" -> ".join(nodes))
```

```

def partition(node, x):
    """
    Partitions a linked list around a value x, such that all nodes less than x
    come before all nodes greater than or equal to x.

    Args:
        node: The head node of the linked list.
        x: The partition value.

    Returns:
        The head of the new partitioned linked list.
    """
    before_start = None
    before_end = None
    after_start = None
    after_end = None

    # Partition the list into two: one for elements < x and one for elements >= x
    while node is not None:
        next_node = node.next
        node.next = None # Disconnect the node from the original list

        # COMPLETE THE LOOP BY INSERTING THE NODE INTO THE END OF
        EITHER THE 'before' LIST OR THE 'after' LIST

        node = next_node

    if before_start is None:
        return after_start

    # Merge the 'before' and 'after' lists

    return before_start

```

Answer:

```

class Node:
    """A simple node for a singly linked list."""
    def __init__(self, data=0, next_node=None):

```

```

        self.data = data
        self.next = next_node

class LinkedList:
    """A simple linked list class for demonstration purposes."""
    def __init__(self):
        self.head = None

    def append(self, data):
        """Appends a new node with the given data to the end of the list."""
        if not self.head:
            self.head = Node(data)
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = Node(data)

    def print_list(self):
        """Prints the elements of the list."""
        nodes = []
        current = self.head
        while current:
            nodes.append(str(current.data))
            current = current.next
        print(" -> ".join(nodes))

def partition(node, x):
    """
    Partitions a linked list around a value x, such that all nodes less than x
    come before all nodes greater than or equal to x.

    Args:
        node: The head node of the linked list.
        x: The partition value.

    Returns:
        The head of the new partitioned linked list.
    """
    before_start = None
    before_end = None
    after_start = None

```

```

after_end = None

# Partition the list into two: one for elements < x and one for elements >= x
while node is not None:
    next_node = node.next
    node.next = None # Disconnect the node from the original list

    if node.data < x:
        # Insert node into the end of the 'before' list
        if before_start is None:
            before_start = node
            before_end = before_start
        else:
            before_end.next = node
            before_end = node
    else:
        # Insert node into the end of the 'after' list
        if after_start is None:
            after_start = node
            after_end = after_start
        else:
            after_end.next = node
            after_end = node

    node = next_node

if before_start is None:
    return after_start

# Merge the 'before' and 'after' lists
before_end.next = after_start
return before_start

```